Programming and Systems Development

Mock Lab Exam A - Python Part (40 marks)

Time allowed for this mock: 1 hour (the same as the real mid-term test). Open book: you may use your printed notes and practice materials. No communication with other candidates. No Tkinter/GUI is needed.

You must write the code for ALL tasks in the SAME Python source file.

Python Task 1: Python Basics and Data Manipulation [13 marks]

You have been provided with a CSV file `cafe_orders.csv` containing the orders taken at a university cafe. The file has the columns `OrderTime`, `Item`, `Quantity`, `Price` and `Total` (all prices are in pounds). `OrderTime` is written like `2024/03/05 09:15` (year/month/day hour:minute). Place the file in the same folder as your program. Your program must:

Task 1a: Data Loading and Preprocessing

- **Load the Data:** load the CSV file into a pandas DataFrame. [2 marks]
- **Data Cleaning:**
  - Ensure there are no missing values. If any are found, fill them with appropriate values. [2 marks]
  - Ensure the data contains no duplicate orders. [1 mark]
  - Convert the `OrderTime` column to a datetime object. [2 marks]
  - Ensure the `Total` column correctly represents the product of `Quantity` and `Price`. If any discrepancies are found, correct them. [2 marks]

Task 1b: Data Visualisation

- **Item Sales Distribution:** create a bar chart showing the total quantity sold for each item. [2 marks]
- **Sales Over Time:** create a line plot to show the trend of total sales over the months of 2024. The x-axis should represent the months and the y-axis the total sales. [2 marks]

Python Task 2: Python Database Management [15 marks]

Problem: managing the cafe orders with SQLite. You are required to create a database to store and manage the order data cleaned in Task 1.

Task 2a: Database Creation and Data Insertion

- Create an SQLite database named `CafeDB`. [2 marks]
- Create a table named `Orders` with appropriate columns to store the data (`OrderTime`, `Item`, `Quantity`, `Price`, `Total`). [3 marks]
- Insert the cleaned and processed data from Task 1 into the `Orders` table. Ensure no duplicate entries are added, even if the program is run more than once. [4 marks]

Task 2b: Querying the Database

- **Total Sales Calculation:** write a query to calculate the total sales for the year 2024 and display the result. [3 marks]
- **Item Sales Summary:** write a query to list each item and its total quantity sold in 2024. Display the result in descending order of quantity. [3 marks]

Example layout of the output (the values here are only placeholders):

```
Total sales in 2024: 000.00

Item sales summary for 2024:
  ItemName     00
  OtherItem    00
```

Python Task 3: Basic Neural Network Implementation [12 marks]

In this task you will define a small fully connected neural network using PyTorch, train it on synthetic data, track the loss during training, visualise it and measure its accuracy.

Network definition: define a class `CafeNN` that extends `torch.nn.Module`. The network has these layers:

- Input: 6 input features.
- Hidden Layer 1: a fully connected layer with 8 neurons and ReLU.
- Hidden Layer 2: a fully connected layer with 4 neurons and ReLU.
- Output Layer: a fully connected layer with 1 neuron and Sigmoid.

Call `torch.manual_seed(42)` once, just before you create the model, so that your results are repeatable.

Task 3a: Model Initialisation [3 marks]

- Instantiate the `CafeNN` model.
- Use `torch.nn.BCELoss()` as the loss function.
- Use `torch.optim.Adam` as the optimiser with a learning rate of 0.01.

Task 3b: Data Generation [2 marks]

Generate synthetic data with 200 samples, each with 6 features, using `torch.randn`. The target is a tensor of 200 values (shape 200 x 1): it is 1.0 when `feature0 + feature1 – feature2 > 0`, otherwise 0.0.

Task 3c: Training the Network [3 marks]

Train the network for 30 epochs. In each epoch compute the loss, perform backpropagation and update the weights. Record the training loss of each epoch.

Task 3d: Loss Visualisation [2 marks]

Plot the training loss over the epochs using matplotlib. Your plot should show the loss decreasing over time, indicating that the model is learning. Give the plot a title and axis labels.

Task 3e: Accuracy [2 marks]

After training, use the model to predict the labels of the training data (treat an output above 0.5 as class 1) and print the accuracy as a percentage, for example `Training accuracy: 00.0%`.

Submission

Ensure your submission file is named exactly as instructed, i.e. `your_name_python_task.py`. Upload that single file to the submission link. Marks are awarded for correctness, and also for clear naming, helpful comments and tidy layout.

Task	Marks
1a Loading and preprocessing	9
1b Visualisation	4
2a Database creation and insertion	9
2b Querying	6
3 Neural network (3a-3e)	12
Total	**40**